VLM Reliability Toolkit — Explained for Friends

VLM Reliability Toolkit — Explained for Friends

A detailed, plain-English guide to the project, plus the technical breakdown underneath.

Part 1: The Big Picture (Plain English)

The Elevator Pitch (one sentence)

It's a **"fact-checker for AI that looks at images."** You show it a picture and ask an AI a question about it, and instead of just trusting the AI's answer, this tool double-checks every claim the AI makes against the actual pixels in the image — then tells you whether to believe it, correct it, or throw it out.

The Problem It Solves

AI models that can see images (called **VLMs** — Vision-Language Models, like GPT-4V, Claude, Gemini, etc.) are impressive, but they **hallucinate**. They confidently say things that aren't actually in the picture:

- "There's a red car" when there's a blue car and a red fire hydrant nearby
- "A person is holding a surfboard" when there's no surfboard
- "Three dogs" when there are two

Most of the time, you have no way of knowing if the AI is right or just making things up — especially if you can't see the image yourself, or if the image is complex.

This project builds a **reliability layer** that wraps around *any* existing image-AI and verifies its answers claim-by-claim, without retraining or changing the AI itself.

How It Works (Step by Step)

Here's the pipeline, using a concrete example:

Step 1: You upload an image and ask a question

You upload a photo (say, a street scene) and type something like: *"Describe the objects and their colors."*

Step 2: The AI generates an answer (multiple times)

The tool asks the connected VLM (OpenAI, Groq, Gemini, Llama, etc.) to answer. It doesn't just ask once — it generates **several candidate answers** (you can pick 1 to 3). This is important because if the AI gives three different answers, that disagreement itself is a signal that something might be uncertain.

Step 3: It breaks the answer into atomic claims

The tool takes the AI's text and splits it into tiny, checkable pieces. For example, if the AI says:

> "A red car is parked next to a blue bicycle."

It extracts claims like:

- **Existence claim:** "There is a car."
- **Binding claim:** "The car is red."
- **Existence claim:** "There is a bicycle."
- **Binding claim:** "The bicycle is blue."

Step 4: It looks at the image to verify each claim

This is the core magic. For each claim, it runs two visual-verification models:

1. **Grounding DINO** — an object-detector that finds where claimed objects are in the image and gives them bounding boxes. If the AI claims there's a car, Grounding DINO looks for a car and says, "Yes, I found one here, with 85% confidence" or "No, I don't see one."

1. **CLIP** — a model that compares image regions against text descriptions. If the AI claims the car is red, CLIP doesn't just check if *anything* in the image is red. It **crops out just the car region** (using the box from Grounding DINO) and checks the color *on that crop*. So if there's a red car *and* a blue bicycle, and the AI says "blue car," CLIP will look at the car crop, see it's actually red, and mark the "blue car" claim as contradicted.

This matters because a naive system might see red somewhere in the image and say "the red car claim is supported." This toolkit is smarter — it checks the right object in the right place.

Step 5: It scores and decides

Each claim gets a status:

- **SUPPORTED** — the visual evidence backs it up
- **CONTRADICTED** — the image shows the opposite
- **UNSUPPORTED** — the object/attribute wasn't found
- **AMBIGUOUS** — the verifier couldn't tell (e.g., a model weight failed to load)

Then it combines:

- Visual verification evidence (60% weight by default)
- Whether multiple candidate answers agree with each other (20%)
- The AI's own confidence in its words (20%, if available)

And produces one of four final decisions:

Decision	What it means
----------	----------
ACCEPT	The claims are supported by visual evidence. Trust the answer.

Decision	What it means
CORRECT	Some claims are wrong, but there's enough evidence to fix them. The tool outputs a corrected version.
RERANK	Multiple candidates were generated; pick the most reliable one after verification.
ABSTAIN	The evidence is too weak or conflicting to make a call. Don't trust or correct — just say "I don't know."

Step 6: Show you everything in a dashboard

There's a web dashboard (Next.js) where you can:

- See the original AI response and any corrected version
- Look at each individual claim and whether it passed or failed
- See bounding boxes drawn on the image (green = supported, red = contradicted, amber = ambiguous)
- Compare multiple candidates side by side
- Export the full analysis as JSON or CSV
- Review past experiments in an "Experiments" history tab

A Concrete Example

Input image: A photo of a person on a beach wearing a **red shirt** and **blue jeans**, with **no surfboard**.

VLM says: *"A person in a blue shirt and red jeans stands on the beach with a surfboard."*

The tool's analysis:

- Claim: "person" → SUPPORTED (a person is detected)
- Claim: "shirt is blue" → CONTRADICTED (the person crop shows red)
- Claim: "jeans are red" → CONTRADICTED (the person crop shows blue)
- Claim: "surfboard exists" → UNSUPPORTED (no surfboard detected)

Decision: CORRECT

Corrected output: *"A person in a red shirt and blue jeans stands on the beach."*

There's even a deterministic demo you can run without any API keys or model downloads that shows exactly this correction (**python3 demo_paper_example.py**).

What Makes It Different

Typical approach	This toolkit
----------	----------
Retrain the AI model to hallucinate less	Doesn't touch the model at all — works with any existing VLM

Typical approach	This toolkit
Trust the AI's own confidence score	Independently looks at the image pixels
Check if a color appears *anywhere* in the image	Checks attributes only on the specific object's region
Gives you a yes/no answer with no explanation	Shows claim-by-claim evidence, bounding boxes, and a full audit trail
Black box	Everything is logged: image hash, candidates, claims, evidence, decisions, corrections

---

The Tech Stack (lightly)

- **Backend:** Python, with separate modules for VLM providers, claim extraction, visual verification, reliability scoring, and correction
- **Visual models:** Grounding DINO (object detection) + CLIP (region attribute checking), both from Hugging Face
- **Frontend:** Next.js dashboard with a dark-mode UI
- **Database:** SQLite for storing experiment history and audit logs
- **Providers supported:** OpenAI, Groq, Gemini, Llama (any OpenAI-compatible endpoint)
- **You need to bring:** Your own VLM API key (the toolkit doesn't supply one)

Honest Limitations (what to tell friends if they ask)

- It's a **research tool**, not a safety-critical system. It can still miss things.
- Right now it mainly verifies **object existence** and **color** binding. Other attributes (material, size, etc.) come back as "ambiguous."
- The claim parser understands a curated set of objects and simple phrasing — it won't parse every possible English sentence perfectly.
- It needs Grounding DINO and CLIP model weights downloaded the first time you run it live.
- No benchmark scores are claimed — it comes with evaluation harnesses, but you need your own labelled data to run them.

How to Try It (if you want to)

1. Install Python dependencies
2. Add your VLM API key to a **.env** file
3. Run **./dev-frontend.sh** (starts both the backend API and the web dashboard)
4. Open **http://127.0.0.1:3000**, upload an image, ask a question, and hit "Run reliability analysis"

5 Or run **python3 demo_paper_example.py** for a quick demo that doesn't need API keys

---

Short Version for Friends

> "It's like giving an image-AI a second opinion. The AI looks at a photo and answers a question, then this tool breaks the answer into tiny claims, checks each one against the actual image using object detection and region-level color analysis, and tells you whether to accept the answer, correct it, pick the best version, or abstain because the evidence isn't clear. It doesn't retrain the AI — it just fact-checks it."

Part 2: Technical Deep Dive

1. The Big Picture (Module Map)

The project is split into pipeline stages, each in its own folder:

Folder	What it does
---	---
vlm.py / **vlm_engine.py**	Talks to the image-AI provider (OpenAI, Groq, Gemini, Llama). Generates candidate answers.
claims/	Takes the AI's text and chops it into small, checkable claims (e.g., "the car is red").
verification/	Looks at the actual image using Grounding DINO (object boxes) + CLIP (color on the object crop).
reliability/	Combines visual evidence, candidate agreement, and model confidence into a 0–1 score.
correction/	If a claim is wrong, surgically fixes the text (changes "blue car" → "red car", removes hallucinated objects).
core/	Shared data structures (**Claim**, **Candidate**) and the pipeline orchestrator (**core/pipeline.py**).
database/	SQLite log of every experiment: image hash, prompt, candidates, evidence, decisions, corrections.
frontend/	Next.js web dashboard where you upload images, run analysis, and inspect results.
evaluation/	Benchmark harnesses (POPE, CHAIR, Winoground, SugarCrepe) — requires your own labelled data.
app.py	Legacy Streamlit UI (a simpler alternative interface).

---

2. The Pipeline (the "Run" button in code)

When you click "Run reliability analysis" in the dashboard, **core/pipeline.py:run()** executes this exact sequence:

`python

1. Generate N candidate answers from the VLM

engine = ProviderVLMEngine(provider)

generated_rows = engine.generate_candidates(image_path, query, candidate_count)

2. For each candidate:

a. Extract atomic claims from the text

claims = extract(text)

b. Verify each claim against the image

evidence = verify(image_path, claims)

c. Compute agreement across candidates

agreement(candidates)

d. Score reliability (0–1) and make a decision (ACCEPT/CORRECT/RERANK/ABSTAIN)

score(candidate, weights, high, low)

correct(candidate)

e. Sort candidates by reliability; possibly rerank to the best one

candidates.sort(key=lambda item: item.reliability, reverse=True)

`

Important detail: The toolkit does **not** use the VLM's native "n-best" logits (which most APIs don't expose anyway). Instead, it calls the model N separate times with slightly different prompts ("Candidate sample 1 of 3", etc.). This is explicitly logged as **repeated_controlled_generation_not_native_nbest** so the results are honest about where they came from.

---

3. Claim Extraction — Turning Text into Checkable Pieces

The **claims/** module parses the VLM's free-text response into **atomic claims**. Each claim has:

- **ID** (stable identifier)
- **Type**: **object** (existence), **attribute** (color/binding), **relation** (spatial), **quantity** (count)
- **Subject**: what the claim is about (e.g., "car", "person")
- **Attribute**: e.g., "color" = "red"
- **Object**: for relations (e.g., "next to a bicycle")
- **Value**: the claimed value (e.g., "red", "3")

Example:

VLM text: *"A red car is parked next to a blue bicycle."*

Extracted claims:

1. **object** — "car" exists
2. **attribute** — car is **red**
3. **object** — "bicycle" exists
4. **attribute** — bicycle is **blue**

Why atomic? Because you can't fact-check a whole sentence at once. You check each piece individually against the image. If the bicycle is actually green, only claim #4 fails — the rest can still be supported.

There's also a **claim graph** (**claims/graph.py**) that turns these into nodes and edges: objects are nodes, attributes/quantities/relations are edges connecting them. This gets visualized in the dashboard so you can see the claim structure.

---

4. Visual Verification — Grounding DINO + CLIP (Crop-Level)

This is the core technical innovation. For every claim, the system doesn't just ask "is this true?" It runs two visual models:

Step A: Grounding DINO — Find the object

- Model: **IDEA-Research/grounding-dino-tiny**
- Role: Open-vocabulary object detection. Given a text query like "car", it returns bounding boxes and confidence scores for where cars are in the image.
- If the AI claims there's a car, Grounding DINO looks for one. No car found? → claim is **UNSUPPORTED**.

Step B: CLIP — Check the attribute on the **crop**

- Model: **openai/clip-vit-base-patch32**
- Role: Region-level attribute probing. **Crucially, it never scores on the full image.**

Why crop-level matters (the red car / blue bicycle example):

Imagine an image with a **red car** and a **blue bicycle**. The VLM says: *"There is a blue car."*

A naive system would detect that blue appears *somewhere* in the image (on the bicycle) and mark "blue car" as supported. That's wrong.

This toolkit:

1. Uses Grounding DINO to find the **car** and get its bounding box.
2. **Crops** just the car region out of the image.
3. Runs CLIP on that crop, asking "is this red? is this blue?"
4. The car crop scores high for **red**, low for **blue** → the "blue car" claim is **CONTRADICTED**, and the verified value is "red".

Each piece of evidence gets:

- **status**: SUPPORTED / CONTRADICTED / UNSUPPORTED / AMBIGUOUS
- **score**: 0.0 to 1.0 confidence
- **boxes**: the bounding box coordinates (drawn on the dashboard image)
- **diagnosis**: e.g., **OBJECT_HALLUCINATION**, **ATTRIBUTE_BINDING_ERROR**, **QUANTITY_ERROR**, **RELATION_ERROR**
- **verified_value**: what the image actually shows (e.g., "red" instead of "blue")

Honesty rule: If Grounding DINO or CLIP weights aren't downloaded, the toolkit reports **AMBIGUOUS** — it never fabricates evidence.

---

5. Reliability Scoring — The Math

reliability/engine.py combines three signals into a single 0–1 reliability score per candidate:

The formula

`

visual_score = (verification × wv + probability × wp + agreement × wa) / (wv + wp + wa)

`

Default weights (from **config/config.yaml**):

- **Visual verification**: 0.60 (60%)
- **Token probability**: 0.20 (20%)
- **Candidate agreement**: 0.20 (20%)

Key subtlety: probability redistribution

Most VLM APIs don't give you genuine token-level confidence. If the provider doesn't supply **probability_available = true**, the configured 0.20 probability weight is **redistributed to visual verification** (so verification becomes 0.80, agreement stays 0.20). The audit record explicitly notes that token probability was unavailable — no fake numbers are invented.

Per-claim scoring

Each claim gets its own reliability value:

- **AMBIGUOUS** or missing evidence → 0.5 (neutral)
- **SUPPORTED** → the evidence score
- **CONTRADICTED** / **UNSUPPORTED** → the evidence score (low score = strongly wrong)

The candidate's overall reliability is the weighted blend above, clamped to [0, 1].

---

6. Four-Way Decision Logic

After scoring, each candidate gets one of four decisions. Here's the exact decision tree from **reliability/engine.py**:

1. **CORRECT** — If any evidence is **CONTRADICTED** with high quality (≥0.8) and a clear diagnosis (**OBJECT_HALLUCINATION**, **ATTRIBUTE_BINDING_ERROR**, **ATTRIBUTE_ERROR**, **QUANTITY_ERROR**), OR a **UNSUPPORTED** object claim with high quality and low score. This means "there's enough evidence to fix this."

1. **ABSTAIN** — If any claim is **AMBIGUOUS** or **UNSUPPORTED**, OR reliability is low with no evidence. This means "I can't tell, don't trust it."

1. **ACCEPT** — If reliability ≥ high threshold (default 0.80) AND nothing is contradicted. This means "the evidence supports the answer."

1. **Otherwise** — ABSTAIN (conservative default).

Reranking (candidate selection)

If you generate multiple candidates (e.g., 3), they're sorted by reliability. If candidate #2 is much better than candidate #1 (≥0.15 reliability gap) and candidate #2 would be ACCEPTED, the decision is upgraded to **RERANK** — meaning "don't trust the first answer; use this better one instead."

---

7. Correction Engine — Surgical Text Fixes

correction/engine.py only changes text when there's **contradictory visual evidence with a clear diagnosis**. It uses regex to surgically edit the string:

Diagnosis	What it fixes	Example
----------	----------	---------
OBJECT_HALLUCINATION	Removes the object phrase entirely	"holding a surfboard" → removed
ATTRIBUTE_BINDING_ERROR	Swaps the wrong color for the verified color	"blue car" → "red car"
QUANTITY_ERROR	Fixes the count	"three dogs" → "two dogs"
RELATION_ERROR	Logged but not auto-corrected (too risky)	—
AMBIGUOUS / **UNSUPPORTED**	Logged as a diagnosis but **not** corrected	—

Conservative policy: Unsupported or ambiguous claims are never deleted or changed — only claims with strong contradictory evidence are touched. If no correction is warranted, **corrected_text** equals the original and the diagnosis is **["NO_STRONG_ERROR"]**.

---

8. Candidate Agreement

reliability/engine.py:agreement() measures how much the candidates agree with each other:

- It normalizes each claim into a key: **(type, subject, attribute, value, object, instance_id)**
- It counts how many candidates make the same claim
- Agreement = average fraction of candidates sharing each claim

If all 3 candidates say "red car," agreement is high (1.0). If one says "red car" and two say "blue car," agreement is lower. This feeds into the reliability score: agreement is a signal that the answer isn't a fluke.

---

9. Audit & Database Logging

Every run is persisted to SQLite (**database/db.py**) with:

- Image hash (SHA-256 of the image bytes)
- The original query/prompt
- Configuration (provider, weights, thresholds)
- All candidates with their text, claims, evidence, scores, decisions
- Corrections and diagnoses
- Latency timings per stage

The dashboard has an **Experiments** tab where you can review past runs, export individual experiments as JSON, or export candidates as CSV. This makes the whole thing reproducible and auditable.

---

10. Frontend Dashboard (Next.js)

The web interface (**frontend/**) is where humans interact with it:

- **Upload** an image (PNG/JPG/WEBP, up to 15 MB)
- Choose provider, number of candidates (1–3), enter a question
- Run analysis → see:
- Original AI response
- Each claim with status (green = supported, red = contradicted, amber = ambiguous)
- Bounding boxes drawn on the image
- Claim graph visualization
- Candidate comparison table (ranked by reliability)
- Original vs. corrected text side by side
- Export buttons (JSON/CSV)
- **Experiments** page: history of all runs
- **Evaluation** page: metrics (precision/recall/F1, Brier score, calibration/ECE) — only works if you connect labelled data

There's also a legacy Streamlit UI (**app.py**) with a dark-themed interface that does similar analysis.

---

11. Evaluation & Benchmarks

evaluation/ contains harnesses for standard VLM hallucination benchmarks:

- **POPE** — yes/no object existence questions
- **CHAIR** — checks if mentioned objects actually exist (CAPTCHA-style)
- **Winoground** — vision-language reasoning
- **SugarCrepe** — fine-grained attribute binding (replaces/swaps/adds words)

Important: These require you to supply your own labelled JSONL data. The repo does **not** bundle datasets or claim any benchmark scores. It's a research tool, not a validated safety system.

One-Sentence Summary for Friends

> It's a plug-and-play fact-checker for image-AI models: it generates multiple answers, breaks them into tiny claims, checks each claim against the actual image using object detection and region-level color analysis, then tells you whether to accept the answer, correct it, pick the best version, or abstain because the evidence isn't clear — all without retraining the AI.

Project location: **/Users/skye28/Documents/ChatGPT/vlm**

Demo command: **python3 demo_paper_example.py**

Dashboard: **./dev-frontend.sh** → **http://127.0.0.1:3000**